

Fable 5 使用心法：从 Prompt 技巧到 Unknowns 管理

Nana 的 AI 娜娜 & Codex

2026 年 7 月 6 日



视频作者/频道：Nana 的 AI 娜娜

发布日期：2026-07-05

视频时长：11:51

视频链接：<https://www.bilibili.com/video/BV1U5MT6sEGe/>

内容导览

一页式结论

AI 编程进入 Agentic Coding 阶段后，关键能力从“写出一个更长的 prompt”转向“管理协作中的 unknowns”。Fable 5 与 Claude Code 代表的强模型可以读仓库、改文件、跑命令并持续推进任务；模型越能自主补全空白，人越需要先暴露盲区、确认关键决策、记录偏离，并在合并前证明自己理解了变更影响。

1. **从 Prompt 地图到真实任务地形**：说明长任务为什么会把未写明上下文变成模型猜测和返工来源。
2. **四类 Unknowns**：区分已写明要求、已意识到的疑问、隐性判断标准和真正盲区。
3. **五个协作方法的后半程**：用反问、源码参考、实施计划和 implementation-notes.md 管理高代价决策。
4. **从交付代码到接管代码**：用说明文档、合并前测验和 Fable 发布视频案例证明人已接管结果。
5. **总结与延伸**：把方法压缩成可复用工作流，并给出适用边界和行动清单。

1 从 Prompt 地图到真实任务地形

1.1 先给结论：模型越强，人越要管理未知

本章的核心结论是：进入 Agentic Coding 阶段以后，协作质量的瓶颈从单纯的模型执行能力，转向人类能否把目标、边界、约束和未知提前说清楚。Fable 5 和 Claude Code 在这里代表一种更长链路的 AI 编程场景：模型可以读仓库、改文件、跑命令、跨文件推进任务，也会在执行过程中不断替人做中间判断。模型能力越强，这些判断越多；人类没有提前管理的空白，也越容易变成返工来源。

本章核心判断

Fable 5 的使用心法超出单个 prompt 技巧。它真正提示的是一种角色变化：人类从“写清一次性指令的人”，变成“管理协作未知、审查关键决策、控制任务边界的人”。Claude Code 这样的 agent 越能自主完成工作，人类越需要先识别哪些地方要说明、哪些地方要提问、哪些地方要留下回头路。

视频开头引用 Thariq 对 Fable 5 的总结：表面上，这是一份工具使用指南，里面有 prompt、原型、实施计划和合并前检查；更深一层，它揭示了 AI 编程正在发生的变化。过去使用 AI 写代码，很多人主要在测试模型：能不能理解需求，能不能找到文件，能不能修 bug，能不能跑通测试，能不能像工程师一样理解项目上下文。到了 Fable 5 这一类更强的协作工具，问题开始转移：工作质量经常卡在人类没有讲清楚的部分。

这里的前提很具体：如果任务短、边界清楚，模型能力就是主要变量；如果任务长、跨文件、牵涉产品判断和历史代码，模型会遇到许多缺口。每一个缺口都要求模型做一次选择。选择本身未必错误，风险在于这些选择可能从未被人类确认。

1.2 从补全代码到代理执行：任务边界为什么变复杂

小任务和长任务的差别，在于模型需要自行决策的数量完全不同。让 AI 改一个函数、补一个测试、写一个脚本，通常效果稳定，因为输入、输出和验收方式相对明确。函数该返回什么，测试该覆盖什么，脚本该读写什么文件，这些边界容易被 prompt 写清。

完整功能、复杂模块和产品页面会改变问题的结构。一个“添加功能”的请求可能同时包含状态结构、错误处理、权限路径、服务层拆分、UI 文案、加载状态、失败提示、兼容逻辑、测试范围和代码组织方式。用户只写了目标，Claude Code 仍然需要把目标落成一组具体工程选择；这些选择一旦进入代码库，就会影响后续审查和维护。

用 Python 任务类比 Agentic Coding

写一个小函数像让助手实现一个明确的 Python 函数：输入参数、返回值和边界条件都能写在函数签名附近。做一个完整功能像让助手修改一个现有 Python 项目：它需要理解目录结构、配置方式、已有异常处理、测试习惯和团队偏好。后者需要的能力已经超出“补全几行代码”，更接近在真实项目里代理执行。

Agentic Coding 的关键特征正是在这里出现：agent 会持续推进任务，产出会超出一段代码建议。它能读上下文，也能制造新的上下文；它能修一个 bug，也能在修 bug 的过程中顺手引入抽象、拆出服务层、改写错误边界。执行链条越长，任务边界越像一个需要共同探明的工程空间。

1.3 Prompt 是地图，代码库是真实地形

prompt 像地图，代码库是真实地形。地图可以标出方向、目的地和大致路线，真实地形包含坡度、泥坑、断桥、旧路、临时围挡和当地人的行走习惯。放到软件工程里，地形就是代码库、业务逻辑、历史包袱、现实约束、团队约定和没有写进文档的维护经验。

这个比喻说明了 prompt 的边界。prompt 可以写清“要去哪里”，却很难穷尽“沿路所有可能遇到的东西”。真实仓库里的组件命名、权限判断、老接口兼容、错误提示规范、测试夹具、发布约束和用户习惯，都会影响实现方式。人类没有明说这些地形，模型就会用训练经验和现场推断补上。

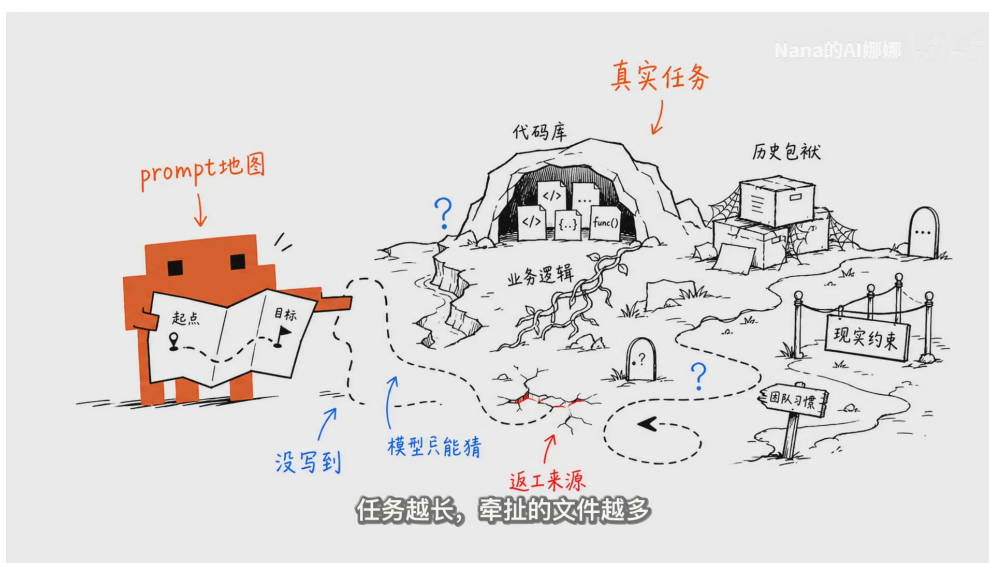


图 1: 视频用“prompt 地图”和“真实任务地形”说明长任务中的代码库、业务逻辑、历史包袱、现实约束和团队习惯。图中蓝色问号和红色裂缝提示，未写出的上下文会成为模型猜测和返工来源。¹

因此，写 prompt 的目标无需变成一次性塞满所有细节的文本工程。更合理的做法是承认地图和地形存在差距：先标出目标，再让 agent 探测地形，报告风险，提出需要人类确认的岔路。后文的 Unknowns 框架、blindspot pass、prototype 和实施计划，都是围绕这个差距建立起来的。

1.4 大任务为什么容易发散

大任务容易发散，核心原因是开放空间里存在太多没有被确认的中间选择。用户说“做一个产品页面”，模型可能决定组件层级、状态放在哪里、数据从哪里来、失败时展示什么、交互动效做到什么程度、样式如何抽象、哪些测试要补。用户说“改一个复杂模块”，模型可能决定是否拆服务层、是否兼容旧路径、是否增加配置项、是否重写错误处理。

这些输出经常看起来很专业：目录变整齐了，抽象变多了，类型更完整了，测试也出现了。专业外观会制造一种错觉，好像实现已经贴近目标。真正需要检查的是：这些选择是否来自明确需求、项目惯例和人类确认。视频中的关键提醒就在这里：模型是在填补人没有说出来的空白；如果空白没有被管理，返工会在后面集中出现。

常见误区：专业外观不等于已确认决策

长任务产出的代码越完整，越需要检查其中哪些选择属于已确认需求，哪些选择只是模型补出来的默认判断。服务层、状态结构、错误提示、权限路径和 UI 行为都可能显得合理；它们只有在项目约束和人类目标下通过确认，才真正可靠。

这也解释了为什么小任务经常惊艳，大任务开始发散。小任务的未知少，模型需要猜的地方少；长任务的未知多，模型必须替人补空白。协作能力的重点因此从“写一个更长的 prompt”，转向“让未知尽早显形”。

1.5 本章小结

本章建立了全文的主线：Table 5 和 Claude Code 所代表的 Agentic Coding，让 AI 从代码补全者变成长期协作者；协作链条越长，未写明的目标、约束和判断标准越会影响结果。prompt 只能提供地图，真实项目地形仍然需要探测。

下一章将把这些空白拆成四类 Unknowns: Known Knowns、Known Unknowns、Unknown Knowns 和 Unknown Unknowns。这个分类的目的在于帮助人类判断哪些内容应该写进需求，哪些问题应该让 Claude Code 先问清楚，哪些风险应该通过 blindspot pass 和 prototype 提前暴露。

2 四类 Unknowns：先承认未知，再扩大视野

2.1 四类 Unknowns 的工作定义

本章的结论是：管理 Agentic Coding 的第一步，是把“我知道什么、我知道自己还不知道什么、我心里有标准却没写下什么、我完全没意识到什么”分开。四类 Unknowns 是实际协作分流工具，它们决定了人类应该写需求、让 Claude Code 提问、做原型，还是先让 agent 扫描风险。

¹ 视频画面时间区间：00:01:12–00:01:31。

English term	中文工作定义	在 AI 编程中的典型含义
Known Knowns	已知且已写明	人类已经知道，也已经明确写进 prompt 的要求。例子包括按钮位置、点击行为、明确接口、固定输出格式。
Known Unknowns	已知但未想清	人类知道这里需要判断，却还没有答案。例子包括是否支持移动端、接口失败如何提示、权限不足时用户看到什么。
Unknown Knowns	心里有标准但未写明	人类有审美、语气、交互和产品判断，却很难提前完整表达。看到结果后，人才会快速判断“太重”“太花”“信息层级不对”。
Unknown Unknowns	尚未意识到的问题	人类连问题本身都没有发现。例子包括陌生领域的坑、代码库历史约定、旧路径兼容、以前的人为什么这样写。

这个表把“未知”拆成四种协作状态。Known Knowns 适合直接写进 prompt；Known Unknowns 适合让 Claude Code 反问；Unknown Knowns 适合通过 prototype 把隐性标准变成可观察对象；Unknown Unknowns 风险最高，适合先做 blindspot pass。



图 2: 视频中的“四类 Unknowns”画面给出四象限结构。下方字幕遮挡了部分 lower-row 文本，因此正文表格复写了完整定义。²

最高风险来自 Unknown Unknowns

AI 编程中最危险的空白，往往集中在人类还没有意识到的问题上。人类没有意识到，就不会写进 prompt；Claude Code 如果直接实施，就会用自己的默认判断填补空白。blindspot pass 的价值，就是先把这类问题从黑暗处拉到桌面上。

2.2 Agentic coder 的能力：熟悉代码库、模型脾气和自己的标准

视频指出，优秀的 agentic coder 强在三种熟悉：熟悉代码库，熟悉模型脾气，熟悉自己想要什么。熟悉代码库，意味着知道哪些模块有历史包袱、哪些接口不能轻易动、哪些测试代表关键行为。

² 视频画面时间区间：00:02:11–00:02:29。

熟悉模型脾气，意味着知道 Claude Code 在模糊需求下容易补哪些默认结构、容易过度抽象哪些地方、在哪些任务上需要更具体的约束。熟悉自己的标准，意味着能区分哪些选择可以交给模型发挥，哪些选择必须由人类确认。

这三种熟悉共同服务于一个目标：减少无意识的委托。把所有选择都交给模型，看起来省事，实际会把审查成本推到最后；把所有细节都写进 prompt，看起来严谨，实际会让人陷入低效的长文本控制。更稳定的做法是按风险分层：普通实现细节可以委托，架构方向、权限边界、兼容路径、用户可见行为和审美判断需要前置确认。

“模型脾气”指什么

这里的“模型脾气”可以理解为 agent 在模糊输入下的默认倾向。比如它可能倾向于补齐完整结构、拆出服务层、写更多抽象、生成看似全面的测试，或者把界面做成常见模板风格。熟悉这些倾向，可以帮助人类判断何时给边界、何时让它探索。

因此，agentic coder 的能力不只体现在 prompt 文采上。更重要的是知道提问顺序：先找最高风险未知，再处理会改变架构的选择，然后再进入具体实现。这个顺序会直接影响返工成本。

2.3 方法一：Blindspot Pass 盲区扫描

Blindspot Pass，也就是盲区扫描，适用于人类要进入陌生模块或陌生领域时。它的 prompt 姿态很简单：先承认自己不熟，再让 Claude Code 查代码库、看已有模式、找风险，最后输出后续提问方向。

Blindspot Pass 的基本 prompt 姿态

“我对这个模块不熟。请先帮我做一次 blindspot pass，扫描这里可能有哪些我没意识到的坑、历史约定、权限路径、兼容要求和测试风险。先不要实现功能，先帮我把后面应该问清楚的问题列出来。”

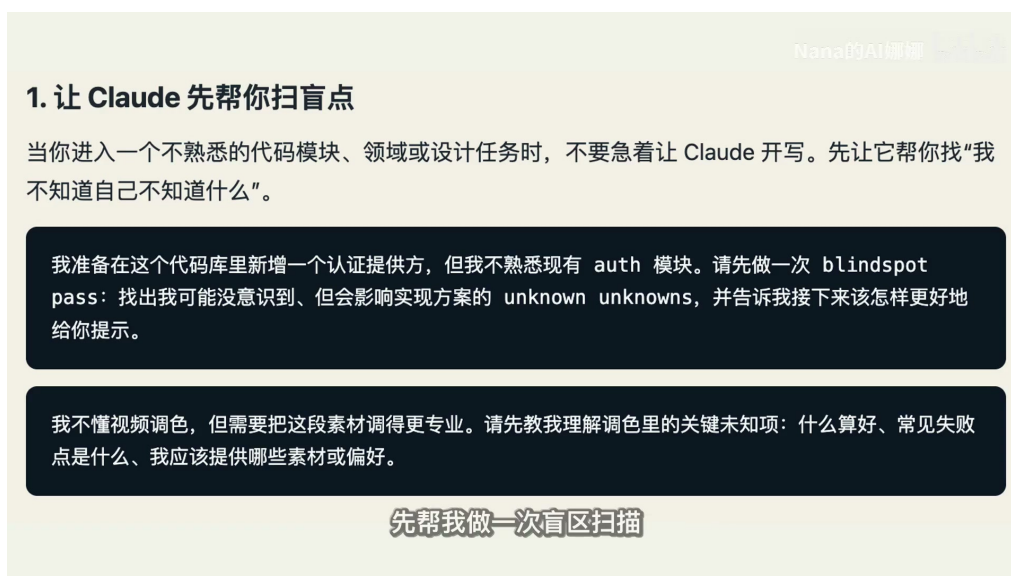


图 3：盲区扫描的画面把方法落成可复用 prompt：先声明自己不熟悉模块，再要求 Claude 找出会影响实现的 unknown unknowns。³

视频给出的例子是接入新的认证方式：用户对现有 auth 模块不熟，却直接要求 Claude Code “帮我接入这个 provider”。这个起点风险很高，因为 auth 模块通常牵涉权限、会话状态、旧 provider 兼容、失败路径、测试覆盖和安全边界。更好的起点是先让 agent 做盲区扫描：现有模块以前怎么处理权限，哪些路径必须兼容，哪些测试不能漏，哪些地方看起来简单、实际会影响架构。

Blindspot Pass 的产物是一份先于最终代码出现的风险地图。它应该帮助人类获得更准的问题：哪些接口要确认，哪些用户状态要覆盖，哪些旧逻辑要保留，哪些实现方案会扩大影响面。此时 Claude Code 的角色从执行者扩展为协作中的侦察者，它先扩大人类视野，再进入实施。

盲区扫描的边界

Blindspot pass 不能替代人类决策。它只能暴露可能的坑、相关文件、历史约定和待确认问题。真正的产品目标、风险接受程度、兼容策略和上线边界，仍然需要人类确认。

2.4 方法二：先做原型，把模糊审美变成可判断对象

第二个方法是 prototype，也就是先做原型。它特别适合界面、设计、交互和信息层级这类 Unknown Knowns：人类心里有标准，却很难在 prompt 里一次讲清。比如“高级一点”“清爽一点”“像某个产品一点”，这些词对人有感觉，对模型却未必构成可执行约束。

更稳的做法是先让 Claude Code 做一个独立 HTML 页面，用假数据搭出几种方向。人看到可交互或至少可观看的对象以后，判断会变快：这个太重，那个太花；这个布局方向对，那个信息层级不行；这个文案接近，那个语气偏离。原型把模糊审美从脑内感觉变成外部对象，让双方围绕同一个东西讨论。

2. 用头脑风暴和原型挖出“我看了才知道”的偏好

Nana的AI课娜

很多判断不是你没有标准，而是你很难提前说清楚。视觉设计、产品体验、数据看板都常见这种情况。先让 Claude 做多个方向的 HTML 原型，再根据你的反应收敛。

我想为这组数据做一个 dashboard，但还没有明确审美方向。请做一个单文件 HTML，给出 4 个差异很大的设计方案，先用假数据，我看完再决定方向。

先不要动真实应用。请用一个 HTML 文件 mock 新编辑器工具栏，数据和交互都可以是假的，我先评估布局和信息密度。

用户在 onboarding 后流失。请先读代码库，列出 10 个可能介入的位置，从低成本到高投入排序，并说明每个方案可能解决什么 unknown。
这种时候，不要急着改真实项目

图 4: 原型 prompt 示例把 dashboard、编辑器工具栏和 onboarding 等模糊判断转成独立 HTML 原型，先用假数据暴露方向差异。⁴

³视频画面时间区间：00:03:37–00:03:56。

⁴视频画面时间区间：00:04:29–00:04:46。

原型成本和真实项目回滚成本

原型阶段的错误通常只影响一个独立 HTML 页面和假数据，删除或重做都很轻。真实项目阶段的错误可能已经接入状态、接口、权限、测试和样式体系，撤回会牵动更多文件。先做 prototype 看起来像绕路，实际是在低成本区域里购买判断力。

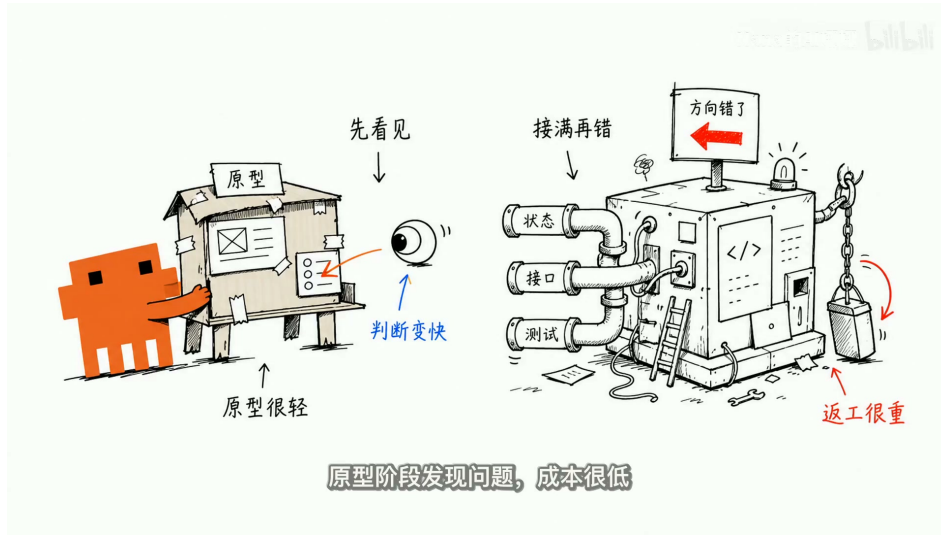


图 5: 视频用“原型很轻”和“接满再错”的对照说明成本差异：先看见方向问题，远比在真实状态、接口和测试接入后再返工便宜。⁵

这个方法还有一个隐藏价值：它训练人类表达标准。人类常常在看见结果之后，才知道自己想要什么。原型提供了一个反应面，让人类把“感觉不对”翻译成更具体的约束，比如“信息层级太平均”“主操作不够突出”“视觉重量集中错了”“移动端首屏负担过重”。这些反馈进入真实实现前，成本最低。

2.5 本章小结

本章把 AI 编程中的空白拆成四类：Known Knowns 需要写清，Known Unknowns 需要提问，Unknown Knowns 需要用 prototype 显形，Unknown Unknowns 需要用 blindspot pass 扫描。这个分类让人类摆脱“把所有问题都塞进一个长 prompt”的低效做法，并根据未知类型选择协作动作。

下一章将接着处理更后半程的方法：让 Claude Code 反过来问关键问题，给它源码级参考，先写实施计划，再用实施笔记记录执行过程中的偏差和理由。前两章解决的是“先看见未知”，后续方法解决的是“让关键未知在动手前被排序和确认”。

3 五个方法的后半程：反问、参考、计划与笔记

本章结论是：blindspot pass 和 prototype 把未知暴露出来之后，Claude Code 协作的核心就转向三件事：先确认会改变架构的选择，给模型足够具体的 source code 参考，再把执行过程中的偏离写成可复盘的 implementation-notes.md。这样处理以后，长任务会通过问题、参考、计划和笔记保留中间判断，最后的 diff 也能被放回可解释的决策链条里。

⁵ 视频画面时间区间：00:04:50–00:05:07。

后半程方法的共同目标

方法三到方法五解决的是同一个工程问题：人的审查精力有限，模型的行动空间很大。协作流程必须把高代价决策提前、把实现语义具体化、把中途判断记录下来。否则，模型看似顺利完成了任务，评审者仍然很难判断它到底替人做了哪些关键选择。

上一章已经说明，原型阶段发现方向问题的成本较低；等到真实代码已经接入状态、接口和测试，再撤回方向性错误，成本会明显升高。因此，本章从“如何继续推进”切入，重点讨论反问、参考、实施计划和实施笔记这四个动作如何降低后续返工。

3.1 方法三：让 Claude 一次只问一个会改变架构的问题

第三个方法的要点很直接：当任务已经有大概方向，但仍有许多模糊处时，让 Claude 一次只问一个问题，并要求它优先提出会改变架构的问题。这里的 Claude 指 Claude Code 这类能够读取代码库、修改文件、运行命令并持续推进任务的 coding agent；“一次只问一个”让人类可以认真判断每个关键选择，“会改变架构”则规定了提问的优先级。

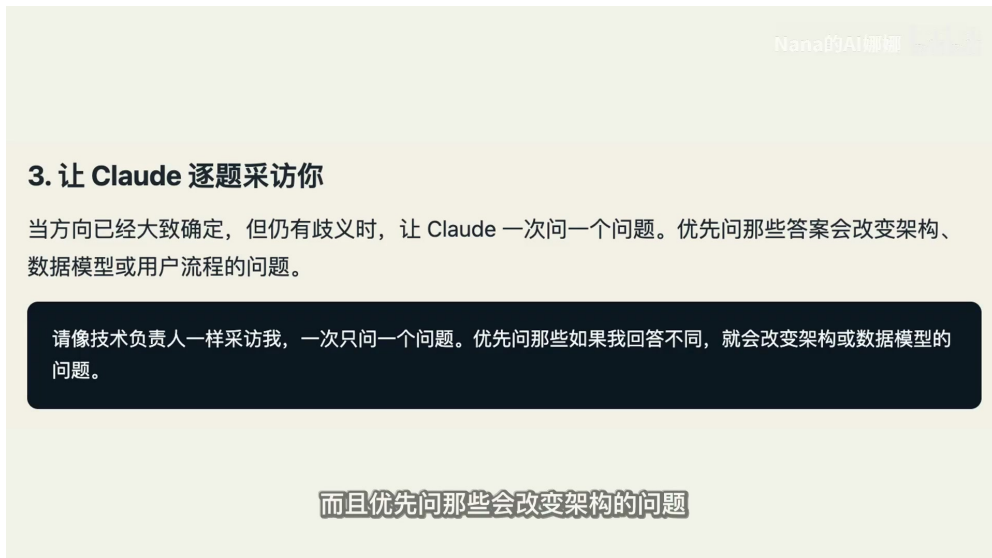


图 6：“逐题采访”画面强调一次只问一个问题，并优先问会改变架构、数据模型或用户流程的岔义。

6

这类问题的价值在于，它们决定后续实现的路线。按钮文案、颜色、局部间距通常可以在后期调整；数据模型、类型接口、权限边界和用户流程一旦被错误确定，后续的文件结构、测试形态、兼容路径和迁移成本都会跟着改变。一个靠谱的工程师接到需求后，会先问使用者、失败场景、合法输入、禁止行为、维护责任和旧数据处理；Claude Code 也应该被要求用同样的顺序做需求澄清。

把问题分层，避免审查预算被稀释

所有问题放在同一层级时，模型可能先讨论按钮名称、页面色彩或微交互细节。这些问题有价值，但它们不应该挤占架构问题的审查窗口。长任务开始前，优先确认数据模型、类型接口、权限边界、用户流程、合法输入、禁止行为、旧数据和维护责任。

⁶ 视频画面时间区间：00:05:11–00:05:28。

问题类别	需要先确认的原因	示例提问
数据模型	决定字段、关系、迁移路径和测试样本	这个功能需要新增实体，还是扩展现有实体？旧数据如何兼容？
类型接口	决定模块边界和调用方约束	新接口由谁调用？哪些字段必须稳定？哪些字段允许后续演进？
权限边界	决定安全约束和失败处理	哪些角色能执行操作？权限不足时应显示什么状态？
用户流程	决定页面状态和错误恢复路径	用户从哪里进入？失败后能否重试？哪些行为必须被禁止？
维护责任	决定实现保守程度和文档粒度	上线后谁维护？团队已有的惯例和回滚路径是什么？

表 1: 架构改变型问题应该优先进入 Claude Code 的反问清单

这个方法背后的假设是：AI 没有消除工程问题本身，只是让这些问题更快浮出来。让模型反问的目标很明确：尽快把真正影响方向的问题摆出来，让人类在改动成本变高之前做判断。



图 7: 提问筛子把“问更多”改成“筛出方向问题”：谁会用、失败时怎么办、哪些输入合法、哪些行为不能发生，都会改变实现方向。⁷

3.2 方法四：给源码级参考，而只给形容词会漏掉实现语义

第四个方法是给参考。很多需求难以用语言完全压缩，例如某个库里的退避策略、某个网站组件的交互、某个工具栏的布局感觉。此时，抽象形容词容易遗漏关键实现语义；更高保真的输入是 source code（源码）、局部实现、已有组件、真实接口调用或测试用例。

需要区分三类参考材料。screenshot（截图）可以让模型看到表面外观，例如按钮位置、视觉层级和布局密度；文字描述可以说明目标、偏好和限制；source code 则能展示结构如何搭建、状态如何流动、边界如何处理、错误如何恢复、命名如何延续、测试如何覆盖。对 AI 编程来说，模型需要学习可执行的实现语义，同时理解视觉外观。

⁷ 视频画面时间区间：00:05:44–00:06:00。

4. 用参考物替代抽象描述

当你描述不清想要什么时，最好的输入通常是参考物。图片、文档、图表都有帮助，但最强的参考物往往是源代码：它能暴露结构、命名、边界、异常处理和真实实现细节。

vendor/rate-limiter 里的这个 Rust crate 实现了我想要的退避语义。请读懂它，再用我们 TypeScript API client 的风格重做同样的行为。

截图只能告诉模型表面长什么样

图 8: 参考物 prompt 用 Rust crate 的 rate-limiter 语义改写 TypeScript API client，说明源码级参考比抽象描述更能传递实现边界。⁸

为什么 source code 比形容词更接近工程事实

形容词描述的是感受，例如“简洁”“自然”“稳健”。source code 描述的是可执行结构，例如状态在哪里保存、错误在哪里捕获、重试由谁负责、边界条件如何分支。前者适合表达方向，后者更适合约束实现。

视频中给出的例子很具体：一个 rate limiter（限流器）怎样处理重试，一个 API client 怎样处理错误，一个组件怎样拆分 component state（组件状态）。这些信息如果只靠自然语言描述，很容易漏掉调用顺序、异常边界、默认值、重试间隔、失败状态和命名习惯；把源码或相邻实现交给 Claude Code，模型更容易复用项目已有的工程语义。

参考类型	能传达的内容	容易遗漏的内容
形容词描述	风格方向、偏好、语气、粗略目标	状态流、错误边界、命名惯例、异常路径
screenshot	视觉布局、信息层级、交互表面	数据来源、组件拆分、边界处理、复用方式
source code	结构、状态流、边界、错误处理、测试入口	需要人类补充业务目标和审美判断

表 2: 参考材料的保真度决定 Claude Code 能学到哪一层信息

因此，给参考仍然需要人类保留判断权。更合理的做法是把 source code、screenshot 和文字说明组合起来：源码约束实现语义，截图约束可见结果，文字说明约束任务目标和不可接受的行为。

⁸视频画面时间区间：00:06:10–00:06:30。

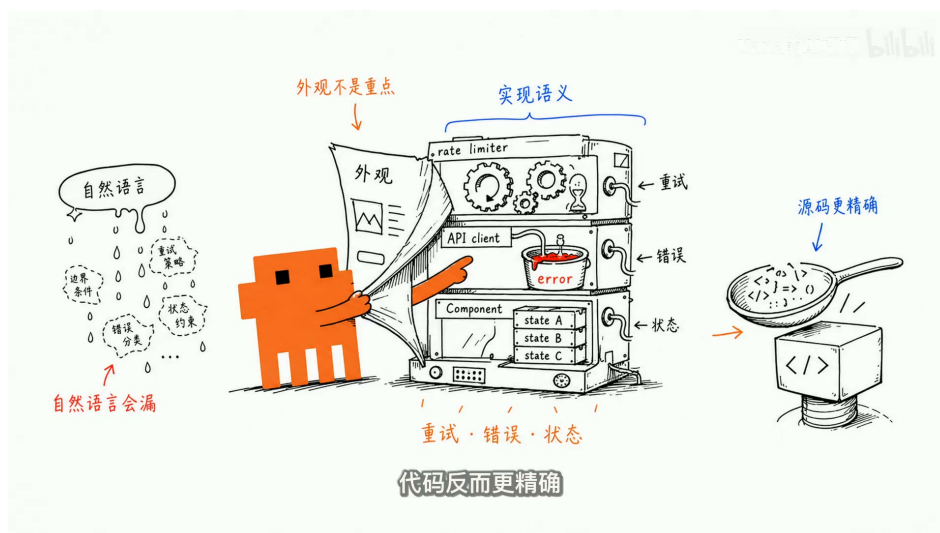


图 9: 视频用“自然语言会漏”和“源码更精确”对照，说明源码能暴露 retry、error、state、边界条件和实现语义。⁹

3.3 方法五：先写实施计划，把关键决策放到最前面

第五个方法是先写 implementation plan（实施计划）。这个计划的价值在于把关键决策提前交给人类审查，并把执行步骤放在决策之后。Thariq 的建议是，让 Claude 把人类最可能想调整的东西放在计划前面，例如数据模型、新的类型接口、用户看得见的流程、迁移路径和很难回头的系统边界。

Nana的AI娜娜

5. 开始写代码前，先让它写可审查计划

实现计划的重点不是列机械步骤，而是把“最可能需要你拍板的地方”前置：数据模型、类型接口、用户可见行为、迁移方案、不可逆决策。

请用 HTML 写一份实现计划。最前面放我最可能要调整的决策：数据模型变化、类型接口、用户可见流程和风险点。机械重构放后面，我主要审前面的判断。

但你必须盯住那些一旦决定就很难回头的部分

图 10: 实施计划 prompt 要求把最可能需要人工调整的决策前置：数据模型、类型接口、用户可见流程和风险点先审查，机械重构放后面。¹⁰

这个排序体现了一个现实判断：人的审查精力有限。长任务中，模型可能生成大量文件、样板

⁹ 视频画面时间区间：00:06:31–00:06:49。

¹⁰ 视频画面时间区间：00:07:00–00:07:19。

代码、测试补丁和格式化变更，人类很难逐行审查所有内容。审查应该先盯住一旦确定就难以回退的部分，再处理机械性重构、文件移动、样板代码和格式调整。

好的 implementation plan 是决策清单

实施计划应该先回答“哪些选择会改变系统形状”，再说明“模型准备怎样执行”。高价值计划会把数据模型、类型接口、权限边界、可见用户流程、迁移方案和不可逆选择放在前面；机械性重构、格式化、文件移动和样板代码可以放在后段。

一个面向 Claude Code 的实施计划可以按下面的顺序组织：

1. 任务目标与成功标准：说明用户最终能看到什么、哪些行为必须成立。
2. 关键决策：列出数据模型、类型接口、权限边界、迁移路径和系统边界。
3. 用户可见流程：说明主要入口、成功路径、失败路径和禁止行为。
4. 参考材料：列出使用的 source code、screenshot、已有 API client、rate limiter 或组件实现。
5. 执行步骤：安排文件修改、测试补充、迁移脚本、文档更新和验证命令。
6. 低风险机械工作：放置格式化、文件搬迁、样板代码补齐等后置任务。

这里的关键指标是计划顺序。计划越早暴露高代价选择，人类越早能校正方向；计划如果把重要决策埋在长列表中，模型仍可能在缺少确认的情况下替人确定系统边界。

3.4 实现中：用 implementation-notes 记录偏离和判断

实施开始后，Thariq 还建议让 Claude 维护一份 implementation-notes.md。它是一份执行期笔记，用来记录模型中途做了哪些决定、什么时候偏离了原计划、为什么选择更保守的方案、遇到了哪些代码边界。它的角色接近轻量级决策日志，服务对象是后续评审者和未来接手代码的人。

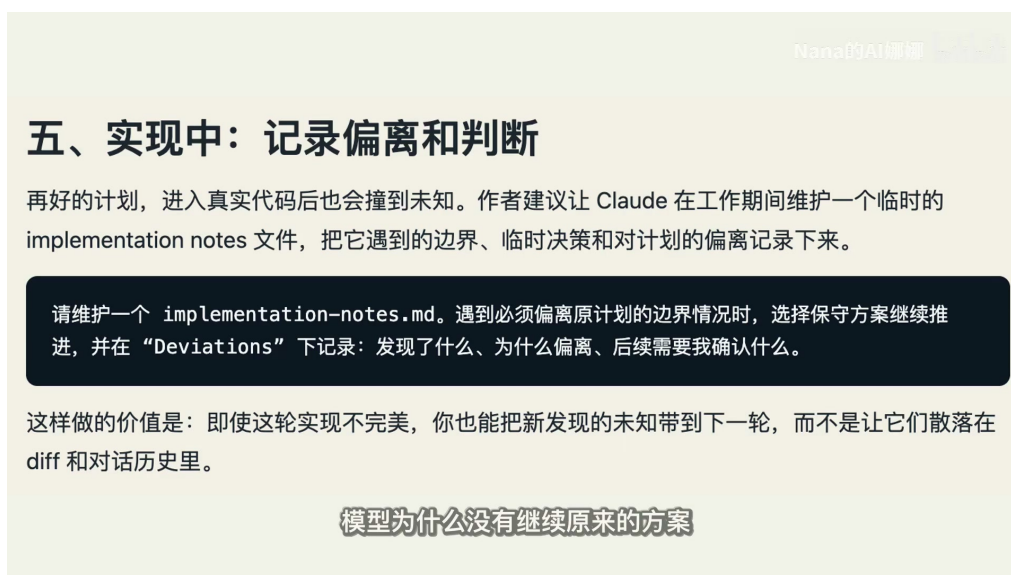


图 11: implementation-notes.md 画面明确要求记录边界、临时决策和偏离原计划的理由，避免新发现只散落在 diff 或对话历史里。¹¹

implementation-notes.md 应该记录四类内容。第一类是中途决策，例如某个接口沿用旧命名，某个状态没有抽到新 store。第二类是偏离计划，例如原计划新增数据表，实际发现复用现有字段更安全。第三类是保守选择的理由，例如迁移逻辑先保持只读兼容，后续再收敛旧路径。第四类是遇到的边界，例如旧数据格式、权限特例、测试夹具缺口或第三方 API 行为。

未记录的决策会让 diff 难以审计

长任务的更高风险来自模型连续做出许多小判断，而人类最后只看到一个 diff。diff 能展示了哪些行，却很难展示为什么偏离计划、为什么保留旧路径、为什么采用更保守的实现。implementation-notes.md 的作用是把把这些判断留在可审计文本里。

一份有效的 implementation-notes.md 可以保持简短，但需要足够具体：

- 记录偏离：原计划哪里不适用，触发偏离的证据是什么。
- 记录判断：模型为什么没有继续原来的方案，选择依据是什么。
- 记录边界：代码里出现了哪些旧路径、兼容约束、权限例外或测试缺口。
- 记录待确认项：哪些判断仍需人类确认，哪些地方需要后续 reviewer 重点看。

这份笔记把“模型为什么这么改”从隐含过程变成显性材料。到了下一章，原型、规格和实施笔记会被进一步打包成说明文档，并用于帮助评审者从同一个上下文理解变更。

3.5 本章小结

本章完成了五个协作方法的后半程。方法三要求 Claude Code 一次只问一个架构改变型问题，帮助人类把审查精力投向数据模型、类型接口、权限边界、用户流程、旧数据和维护责任。方法四强调 source code 参考，因为源码能传达 screenshot 和形容词难以覆盖的实现语义，例如 rate limiter 的重试策略、API client 的错误处理和组件状态拆分。方法五要求先写 implementation plan，并把高代价决策放到计划前面。实施过程中，implementation-notes.md 继续记录偏离、边界和保守选择的理由。

这些动作共同把 Agentic Coding 的中间过程变成可审查材料。下一章要解决的是交付后的接管问题：当 Claude Code 已经完成代码，人类和评审者如何通过说明文档、合并前测验和具体案例确认自己理解了这次变更。

4 从交付代码到接管代码：说明文档、测验和 Fable 案例

这一章的结论是：Agent 写完代码并不等于人已经接管了代码。长任务的交付物至少包含三层内容：可运行的实现、能解释实现路径的说明材料、以及能证明人理解影响范围的合并前测验。只看最后的 diff，只能看到文件和行的变化；要真正接管一次 Agentic Coding 结果，人还要说清楚它为什么这样改、改动碰到了哪些旧逻辑、哪些判断曾经改变过方向。

¹¹ 视频画面时间区间：00:07:40–00:07:58。

4.1 把原型、规格和实施笔记打包成说明文档

实施结束后的第一件事，是让 Claude 把前面留下的原型、规格和 `implementation-notes.md` 整理成一份能给别人看的说明文档。这里的“说明文档”承担的是上下文复原功能：把评审者带回同一个问题现场，说明这个任务原来要解决什么，哪些地方曾经不确定，Agent 如何探索这些不确定，最终实现为什么选择这一条路径。

一个可用的说明文档通常要回答四类问题：

- **问题背景**：这次修改服务于什么目标，原来的行为或体验有什么限制。
- **探索路径**：是否做过原型，原型暴露了哪些视觉、交互或技术判断。
- **实现决策**：哪些数据结构、接口边界、权限路径、状态流转被明确选定。
- **偏离记录**：实际执行中哪里没有沿用原计划，原因是兼容旧逻辑、降低风险、遇到边界条件，还是发现了新的约束。

这份材料的价值在于降低评审者的进入成本。假如评审者和作者最初有同样的 blind spot，说明文档能让他快速理解上下文；假如评审者是领域专家，`implementation-notes.md` 也能让他判断这次实现是否考虑过常见失败点，避免把一整段工作误读成凭感觉推进。



图 12: 说明文档 prompt 要求把原型、规格和 implementation notes 打包成可发给评审者的 explainer，先展示 demo 和价值，再解释关键决策、风险和待审批点。¹²

说明文档像交接单

把 Agent 的交付结果想成一次工程交接。代码是机器本身，diff 是零件清单，说明文档则是维修记录和风险提示。只拿零件清单接班，接手者很难知道哪些零件是关键件、哪些替换是临时权衡、哪些旧路径仍然需要盯住。

说明文档还会反过来约束 Agent 的工作方式。前面若没有原型、规格、计划和实施笔记，结束时就很难整理出高质量解释。也就是说，好的“交付后说明”依赖好的“交付中记录”。这正是 Thariq

¹² 视频画面时间区间：00:07:58–00:08:16。

要求 Claude 维护实施笔记的原因：长任务中最难复盘的部分，往往集中在中途那些没有写下来的判断。

4.2 合并前让 Claude 反过来考你

第二个动作是 self-quiz：让 Claude 在合并前反过来考人。视频里给出的规则很明确，Claude 先给出本次改动报告，再出一套关于影响范围的测验；只有人能完全答对，才说明自己真正理解了这次变更，具备合并资格。

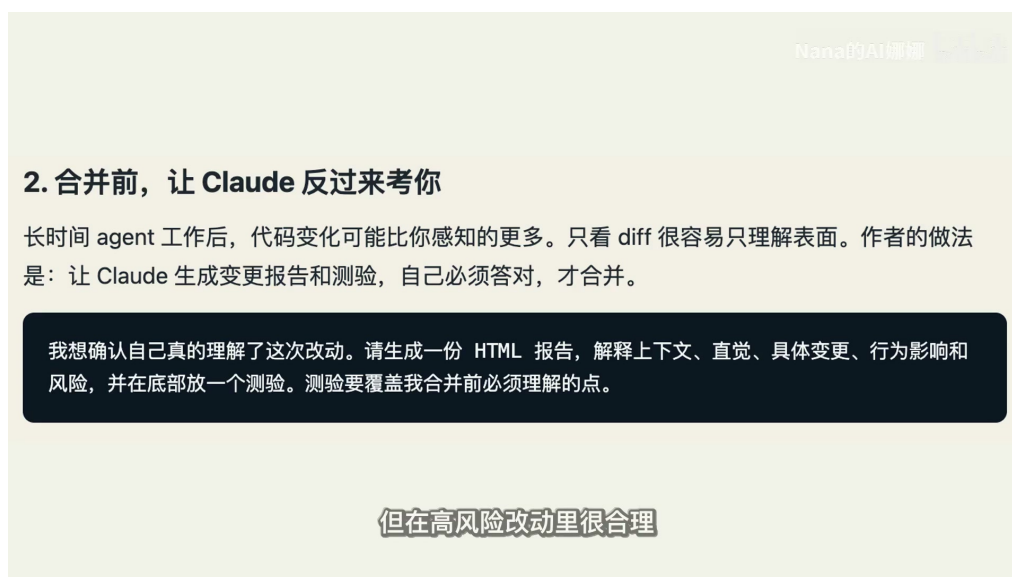


图 13: 合并前自测画面要求 Claude 生成 HTML 报告和测验，覆盖上下文、直觉、具体变更、行为影响和风险。¹³

这个动作针对的是长任务里最容易被低估的风险：很多影响不在新增代码表面，而藏在旧逻辑、旧路径、旧状态里。一个 diff 可以显示新增了哪些文件、删改了哪些行，却很难直接告诉读者：旧权限路径是否还有效，旧数据是否会被迁移，旧 UI 状态是否会触发新的边界条件，失败时用户会看到什么。self-quiz 的目的就是把这些隐性影响变成必须回答的问题。

只看 diff 的浅层风险

长任务里，diff 容易给人一种“已经看过”的错觉。真正高风险的问题常常是旧行为是否被改变、旧状态是否仍可恢复、旧路径是否仍被测试覆盖。人如果答不出这些影响问题，就还没有接管这段代码。

一套有效的 self-quiz 不应该只问“改了哪些文件”。它更适合问下面这些问题：

- 本次改动改变了哪些用户可见行为？
- 哪些旧逻辑、旧路径或旧状态会受到影响？
- 新的数据模型、类型接口或权限边界为什么这样设计？
- 哪些测试覆盖了关键路径，哪些风险仍然需要人工确认？

¹³ 视频画面时间区间：00:08:39–00:08:56。

- 如果线上出现异常，最可能的回滚点和排查入口在哪里？

这类问题把合并从“代码看起来没问题”提升为“人能够解释影响”。它也改变了 Claude Code 的角色：Claude 除了提交实现，还可以在合并前通过提问帮助人建立理解。

4.3 案例：Fable 发布视频如何被 Claude Code 剪出来

Fable 发布视频的案例，把前面的方法从普通代码功能扩展到了视频制作。Thariq 缺少专业剪辑判断，所以没有直接要求 Claude Code “剪一个好看的视频”。他先把自己已知和未知分开：他知道 Claude 可以用代码处理视频、转录文本和 `ffmpeg`；他不确定转录精度、同步界面可行性和调色判断标准。

这个案例可以拆成三段低成本探索。

Nana的AI娜娜

七、案例：Fable 发布视频

作者提到，Fable 的发布视频是用 Claude Code 完成剪辑的，而他本人并不是视频专家。这个过程正好展示了 unknowns 工作法：

- ☐ 先确认自己已知的部分：Claude 能用代码处理视频、转写文本、调用 `ffmpeg`。
- ☐ 再让 Claude 解释不熟悉的技术：转写如何工作，能否稳定剪掉停顿和口头填充。
- ☐ 用 Remotion 和转写稿先做原型，验证画面能否和讲话节奏同步。
- ☐ 发现画面“灰、闷、不够好看”后，不是直接要求 Claude 乱试，而是先让 Claude 教他理解调色标准。

这说明：强模型不是让人跳过思考，而是让人更快找到该思考什么。

图 14: Fable 发布视频案例把 unknowns 工作法落到四个动作：确认已知能力、先解释不熟悉技术、用 Remotion 做原型、先学习调色标准。¹⁴

不确定点	先做的动作	解决的判断问题
转录精度	让 Claude 先解释 Whisper 这类转录技术，并评估能否配合 <code>ffmpeg</code> 剪掉语气词和长停顿	人先理解转录与剪辑边界，再判断自动化剪辑是否值得继续推进
同步界面	用 Remotion 和转录文本做原型	先验证“界面能否跟说话节奏同步”，再决定是否进入正式制作
调色质量	先让 Claude 生成几个选项，随后发现自己缺少判断标准，于是让 Claude 先教调色	从“选哪个版本”退回到“如何判断什么是好调色”

表 3: Fable 发布视频案例中的三类未知与探索动作。

这里的关键点是，Thariq 没有把所有未知都压到最后成片里。Whisper 代表语音转文字和时间对齐的能力边界；`ffmpeg` 代表可脚本化的视频处理；Remotion 代表能用代码生成或验证视频界面

¹⁴ 视频画面时间区间：00:09:40–00:09:57。

的原型工具。每一个工具都被放在“先学习或先验证”的位置，避免直接被当成最终答案生成器。

Fable 案例的核心模式

当人缺少领域判断时，Agent 的第一项价值是帮助人建立判断标准：先解释原理，先做原型，先给可比较选项，先暴露自己不知道如何判断的地方。最终成品应当建立在这些判断标准之后。

调色这一段尤其说明问题。视频看起来“有点发闷”只是一个模糊感受；如果人不知道什么叫好的调色，让 Claude 生成再多版本也只是在增加选择负担。让 Claude 先教调色，等于把 Unknown Unknowns 转成 Known Unknowns：人开始知道自己缺少什么判断标准，也知道接下来该看哪些维度。

4.4 从效率问题回到判断问题

这套做法表面上增加了步骤：先解释 Whisper，先试 Remotion 原型，先学习调色，再继续剪视频。它看起来像绕路，实际是在把后期昂贵返工提前变成低成本探索。真实项目里，越晚发现判断标准缺失，返工越贵；越早用原型、报告和测验暴露未知，越容易把风险限制在小范围内。

视频在这一段给出了人机分工的核心句：模型负责搜索、生成、尝试、迭代；人负责定义目标、暴露未知、判断取舍、控制边界。这个分工适用于代码，也适用于视频这种跨工具任务。Agentic Coding 的成熟形态，是让模型把可搜索、可生成、可试错的部分高速推进，同时让人把目标和边界固定住。

人机分工

模型负责搜索、生成、尝试、迭代；人负责定义目标、暴露未知、判断取舍、控制边界。强模型扩大了执行半径，也放大了未确认假设的风险。

因此，效率的真正来源是让每一步发生在成本最低的位置。原型阶段发现同步做不出来，损失只是一个小样；正式代码接了状态、接口和测试以后才发现方向错了，撤回成本就高得多。合并前测验也是同样逻辑：在合并前发现人没有理解影响范围，成本只是继续追问；合并后才发现旧路径被破坏，成本就是线上修复、回滚和信任损失。

4.5 本章小结

本章把 Agentic Coding 的交付标准从“写完代码”推进到“人能接管代码”。说明文档让评审者和作者站到同一个上下文里；self-quiz 检查人是否理解了旧逻辑、旧路径和旧状态的影响；Fable 发布视频案例说明，Agent 在陌生领域的价值常常是先帮助人建立判断标准。下一章会把这些动作压缩成一套可复用工作流：先暴露未知，再低成本探索，随后让模型在已审查边界内执行。

5 总结与延伸：成熟协作的基本规律

这一章的结论是：成熟的 Agentic Coding 要把隐藏的 unknowns 尽早变成可讨论、可验证、可记录的协作对象。Fable 5、Claude Code 和其他越来越强的 AI 编程工具都会遇到同一个规律：模型越能顺畅推进，人越要主动管理目标、边界、判断标准和合并条件。

5.1 讲者的收束：强模型让未确认假设更危险

讲者在最后的实质性收束中强调，强模型最大的风险来自顺畅推进。它能一路向前推进，能生成大量代码，也能把一个模糊需求补成完整系统；这种智能感会掩盖一个事实：完整系统里可能混入很多人没有确认过的假设。

因此，以后写代码时，语法仍然重要，但只会写语法已经不够。更关键的能力，是知道什么时候该让模型动手，什么时候该先停下来把问题问清楚。遇到陌生模块，先让模型看哪里可能有坑；做界面时，先搭几个能看的小样；遇到会影响架构的选择，先让 Claude Code 反问几个关键问题；代码写完以后，也要确认自己真的明白它改了什么。

这套方法对 Fable 5 有用，对 Claude Code 有用，对任何越来越强的 AI 编程工具也有用。原因很直接：它讲的是人和 Agent 协作时的基本规律。地图永远不等于真实任务，prompt 只能覆盖一部分上下文，写下来的需求也只是人在当时对问题的理解。成熟的 AI 编程，需要人和模型一起把隐藏未知尽早摆到台面上，等看清楚以后，再让模型往前跑。

强模型的隐性危险

模型弱的时候，人会自然盯紧它，因为错误容易被看见。模型强的时候，危险来自顺畅推进：它可以把空白补完整，也可能把未确认假设一起写进系统。

5.2 一张可复用的 Agentic Coding workflow

把全片方法压缩成一条 workflow，可以得到八个连续动作。它们构成控制未知成本的顺序：先发现未知，再让未知变小，随后让 Agent 在可审查的边界里执行，最后确认人是否真的接管结果。

顺序	动作	目的
1	blindspot pass	在陌生模块或陌生领域里，先让 Claude 扫描风险、历史约定、兼容路径和缺失测试。
2	prototype	用低成本原型把模糊审美、交互或可行性问题变成可观察对象。
3	一次问一个架构问题	让 Claude 优先追问数据模型、类型接口、权限边界、用户流程等高代价选择。
4	给源码级参考	用 source code、API client、rate limiter 等真实实现材料传递结构、状态流和错误处理语义。
5	前置实施计划	把最可能需要人调整的决策放到计划前面，把机械重构和样板代码放到后面。
6	维护实施笔记	用 implementation-notes.md 记录中途决策、偏离原因和保守选择。
7	打包说明文档	把原型、规格和实施笔记整理成评审者能快速进入上下文的 explainer。
8	self-quiz 后再合并	让 Claude 出报告和测验，确认人能解释影响范围后再 merge。

表 4: 可复用的 Agentic Coding workflow。

这张表背后的结构很重要。前四步主要解决“我还不知道什么”：盲区扫描扩大视野，原型把模糊判断变具体，反问把高代价选择提前，源码参考降低自然语言描述的损耗。后四步主要解决“我如何安全接管”：计划让关键决策先接受审查，笔记保留执行中的判断，说明文档帮助别人复盘，self-quiz 确认人已经理解影响。

八、一套可直接复用的工作流	
第一步	说清楚你的起点：你熟悉什么、不熟悉什么、现在希望 Claude 扮演执行者、顾问还是采访者。
第二步	做 blindspot pass：让 Claude 找出可能影响成败但你尚未意识到的问题。
第三步	用 HTML 原型或多个方案，把 unknown knowns 变成可以看、可以比较、可以反馈的对象。
第四步	让 Claude 逐题采访你，优先解决会改变架构、数据模型、用户体验的歧义。
第五步	提供参考代码、参考产品、参考设计或参考文档，减少抽象描述带来的偏差。
第六步	正式实现前审一份计划，重点看决策点，不把注意力浪费在机械步骤上。
第七步	实现中记录 deviations，把新发现的未知沉淀下来。
第八步	也只是你当时对问题的理解，正理解发生了什么。

图 15: 视频最后用八步表格总结可复用工作流。截图底部第八步被字幕遮挡，正文表格已复写完整内容并补足“用自己的语言确认理解”这一收束动作。¹⁵

工作流的压缩版本

先让模型帮人发现未知，再用原型和反问降低未知成本；执行时记录关键判断，交付后用说明文档和 self-quiz 证明人已经接管结果。

5.3 适用边界：哪些任务最需要这套方法

这套方法最适合高不确定性、高回滚成本、跨文件或跨领域的任务。典型场景包括陌生模块、auth 或权限变更、UI 与交互设计、数据模型变化、迁移路径、高风险重构、多文件 Agent 任务，以及任何会让模型替人决定系统边界的工作。

在这些任务里，人的主要风险是无法提前识别哪些决定会变贵。例如权限边界选错，会影响安全和用户可见行为；数据模型选错，会影响迁移和旧数据；交互方向选错，会让已经接入状态、接口和测试的 UI 返工；高风险重构如果没有说明文档和测验，评审者只能在巨大 diff 里猜测意图。

低风险小任务对这套流程的需求较低。明确的一行修复、小范围文案调整、可轻松回滚的样式修正、边界清晰的测试补充，通常不需要完整的盲区扫描和 self-quiz。成熟协作的判断力也体现在这里：流程要服务风险密度，避免给所有任务增加同等重量。

用风险密度决定流程重量

任务越陌生、影响面越广、回滚越贵，越应该增加盲区扫描、原型、计划、笔记和测验。任务越小、边界越清楚、回滚越简单，流程就可以更轻。

5.4 给读者的行动清单

读者在启动一次 coding agent 任务前，可以用下面这份清单快速检查协作质量。

- 写下目标：本次任务要改变什么用户行为、系统能力或维护状态。

¹⁵ 视频画面时间区间：00:11:23–00:11:41。

- 标出 unknowns：哪些需求已明确，哪些问题已知道但未想清，哪些标准只存在于直觉里，哪些领域可能有 Unknown Unknowns。
- 先做 blindspot pass：让 Claude Code 检查陌生模块、历史约定、权限路径、兼容要求和测试缺口。
- 遇到界面、交互、视频、视觉质量等模糊判断时，先做 prototype。
- 让 Agent 一次只问一个会改变架构的问题，并优先处理数据模型、类型接口、权限边界、用户流程。
- 给 source-code reference；让模型从源码中看到结构、状态、错误处理和边界，减少形容词或截图造成的语义损耗。
- 要求 implementation plan 把不可逆或难回头的决策放在前面。
- 执行中维护 implementation-notes.md，记录偏离计划的原因。
- 结束后生成 explainer，让评审者从同一上下文理解问题和风险。
- 合并前做 self-quiz；如果答不清影响范围、旧路径和回滚点，就继续追问。

这份清单的核心是让人始终掌握判断权。Agent 可以更快地搜索、生成、尝试和迭代；人的责任是把目标、边界、取舍和风险判断留在可检查的位置。

5.5 开放问题

这期视频给出的是一套实践框架，仍然保留几个开放问题。

- 原始 Thariq 长文中是否还有更细的 Fable 5 操作细节，需要结合原文才能完整说明。
- Fable 5、Claude Code 与其他 coding agent 的具体能力差异，会如何改变 blindspot pass 和 self-quiz 的设计。
- 团队政策、代码所有权和安全要求不同，是否需要把实施笔记、说明文档和测验变成强制合并门槛。
- 当 Agent 越来越能自己运行测试、修复失败并重构大范围代码时，人应该用什么信号校准信任。
- 对新手而言，怎样区分“应该让 Claude 先教我判断标准”和“可以直接让 Claude 执行任务”。

这些问题不会削弱视频的核心结论，反而说明 Agentic Coding 是一套仍在演化的工程协作能力。工具越强，协作协议越重要；模型越能自主执行，人越要把关键假设变成显性材料。

5.6 本章小结

成熟的 AI 编程以 unknowns 管理为中心。prompt 是地图，真实任务是地形；地图只能表达当下理解，地形还包含代码库、旧逻辑、团队习惯、风险边界和人的隐性标准。Fable 5 与 Claude Code 这类工具扩大了模型的执行半径，也要求人更早暴露未知、更清楚定义目标、更严格确认合并前理解。实用的行动原则可以压缩成一句话：先把隐藏假设摆到台面上，再让强模型在已审查边界内快速前进。